TIVIDY: Workflow Management System

COS 214 Practical 6, Task 2: Scope and Scenario

1. Initial Scope of TIVIDY

1.1 Purpose

TIVIDY is a generic workflow management system. It stores workflow definitions, creates independent running workflow instances from them, and manages those instances from start to a final outcome. It must be able to run many different processes without any change to its core. A process such as insurance claims, onboarding or software delivery is supplied to TIVIDY as data and configuration; it is never built into the engine.

1.2 Main Responsibilities

Responsibility	What TIVIDY must do
Definition management	Store versioned workflow definitions that describe stages, work items, their order, dependencies and rules.
Instance creation and execution	Create an independent instance from a definition and move it through its stages until it completes, fails or is cancelled.
Work structure	Represent work as stages, individual work items and sub-processes, run sequentially or in parallel, and let the rest of the system navigate it without knowing how it is stored.
Work-item lifecycle	Track each item through created, available, assigned, in progress, awaiting approval, completed, rejected, cancelled and escalated, and refuse operations that are invalid in the current state.
Assignment and routing	Decide who receives each item and which work becomes available next, using rules that the organisation chooses.
Approvals and escalation	Collect approval decisions (one person or several levels), handle rejection, and escalate when nobody responds in time.
Events and reactions	Publish what happens (item completed, approval requested, external event) so that interested parts can react without the core knowing them.
External integration	Communicate with outside systems whose interfaces were not designed for TIVIDY.
History and recovery	Record every action and state change, and support undo and rollback to a saved checkpoint.
Monitoring and reporting	Produce progress, workload and delay information from recorded activity, without reporting logic inside the core model.

1.3 Boundaries

In scope: definitions, instances, work structure, lifecycles, assignment, approvals, escalation, events, integration points, history, rollback and reporting, as listed above.

Out of scope: the content of any particular process; the business work itself (TIVIDY coordinates writing code or reviewing a document, it does not do it); user-interface design; real authentication, networking and persistent storage (these are simulated or kept in memory for the demonstration); and the internals of external systems.

1.4 Definition Data versus Instance Data

A change to one running instance must never change other instances created from the same definition. The two kinds of data are therefore kept apart.

	Belongs to the workflow definition	Belongs to a running instance
Identity	Name and version number	Unique instance id and title
Structure	Stage templates, item templates, execution mode (sequential or parallel), declared dependencies	The live tree of stages and work items created from those templates
Progress	None	Current state of every item, instance status, completed and pending work
People	Roles that are required	The actual participants who were assigned, plus their workload
Data	Expected payload fields and constraints	The actual payload values entered while running
Rules	Which assignment and approval rules apply	The rule objects in use, approval requests and their decisions
History	None	Audit log, command history and saved checkpoints

Definitions are versioned so that instances already running keep the version they started with, and later edits create a new version instead of altering live work.

1.5 Parts Expected to Vary

What varies	Examples the design must allow
How work is assigned	By role, by least workload, or by a rule calculated from workflow data
How approval works	One approver, several levels, automatic approval for low-risk work, escalation after a timeout
Extra behaviour on a task	Validation, priority handling, auditing, notification or security, added without creating a new kind of task
How a process is built	Different ways of constructing the same definition (programmatically, from an outline)
Organisation policies	A small team and a large enterprise apply different default rules to the same process
Who reacts to events	Activity feed, notifications, metrics, automation rules; new listeners can be added later
External integrations	Different outside systems with different interfaces behind one common connection point
How the work tree is viewed	Traversal in depth-first order, or only the work that is ready now
Reports and views	New reports should not require changes to the work-item classes

1.6 Design Rationale

The scope above follows what the research found repeating across existing workflow products. Trello and Jira differ in size and style, but both separate the same few concerns, and TIVIDY draws its boundaries along those same lines.

Scope decision	Reasoning from the research	Source
Engine separate from process definitions	Jira reuses workflows and settings across projects, and Trello reuses board templates. The engine stays fixed while each process is saved as a definition. This is why TIVIDY stores definitions and runs the same engine for any process.	Atlassian (2024); Atlassian (2022)

Scope decision	Reasoning from the research	Source
Fixed core, configurable behaviour	In both products the core structure (item, state, move) is fixed, while permissions, notifications, updates and rules are configurable. TIVIDY therefore fixes the work structure and lifecycle, and leaves rules to the organisation.	Atlassian (2024)
Definitions versioned; instances keep their version	Jira saves edits as drafts and publishes them as new versions, and running issues finish on the version they started with. TIVIDY copies this so that changing a definition never alters live work.	Atlassian (2024)
Roles, not named people, in definitions	Jira assigns permissions through roles and groups, and conditions limit who may perform a transition. TIVIDY definitions name required roles, and the actual participants are chosen on each instance.	Atlassian (2020); Atlassian (2025)
Assignment rules vary by organisation	Jira picks an assignee by an ordered set of rules or by round-robin, and Trello can assign by rule. How work is assigned is therefore a decision to be swapped, not fixed in the work item.	Atlassian (2019)
Approval is a decision with two outcomes; escalation is a separate timer	In Jira an approval has one path for approve and one for decline, while escalation is timer-based (for example when an SLA is about to breach). TIVIDY keeps these separate so each can change independently.	Atlassian (2024)
Events, history and integrations kept outside the core	Both products send events out through webhooks, accept outside calls through a REST API, and record history separately from the work. TIVIDY keeps these concerns out of the core model so new listeners and connectors can be added without redesign.	Practical 6 brief
Scenario kept outside the engine	The brief requires the engine to be generic and the scenario to be one application of it. Software delivery therefore appears only in the definition, the roles and the external connector, as shown in Section 3.	Practical 6 brief

The boundaries also keep the design focused on its purpose. Persistence, networking, authentication and user interface are left out because Practical 6 is about the structure and behaviour of the workflow model; they can be simulated for the demonstration without changing that model.

2. Selected Scenario: Software Feature Delivery

The scenario below is one application of TIVIDY. Nothing in the generic engine refers to features, code or releases; those concepts exist only in the workflow definition and in the connector to an external system.

2.1 Organisation or Environment

A mid-sized software company with a product team of about ten people. The team delivers new features for a web application. Every feature goes through the same defined process, but the rules differ with the feature: high-risk features, such as changes to payments or login, need stricter review than small interface changes.

2.2 Process Being Managed

Taking a single approved feature request from intake to a released and documented feature. Each feature is one workflow instance created from the "Feature Delivery" definition.

2.3 Participants and Roles

Role	Typical participant	Responsibility in the workflow
Requester	Customer or internal stakeholder	Raises the feature request; accepts the result.
Product Owner	Product team member	Clarifies requirements and sets priority and risk level.
Developer	Engineer	Implements backend, frontend and tests.
Reviewer	Senior engineer	Reviews code and approves or requests changes.
QA Tester	Quality engineer	Verifies the feature against its acceptance criteria.
Tech Lead	Engineering lead	Second-level approver; receives escalations.
Release Manager	Operations	Gives final release approval and schedules deployment.
Git hosting service	External system	Holds branches, pull requests and merges (not a human participant).

2.4 Major Pieces of Work

The process has five stages. Stages run in order; work inside a stage may be parallel.

- **1. Intake and Planning:** Write acceptance criteria, estimate size, set risk level and priority. (sequential)
- **2. Implementation:** Implement backend, implement frontend, write documentation. (parallel)
- **3. Review and Verification:** Code review and QA testing. Security review is added for high-risk features. (parallel)
- **4. Release Approval:** Tech Lead approval, then Release Manager approval. (sequential)
- **5. Release and Close-out:** Merge, deploy, announce, and close the request. (sequential)

2.5 Important Decisions and Routing Points

- **Who gets each item:** A small team assigns by role; a busy team assigns to the least-loaded Developer; high-risk work is routed by a rule based on the risk value in the payload.
- **Risk routing:** Is the feature high-risk? If so, an extra security review item is added to stage 3.
- **Code review outcome:** Approved goes on to QA; changes requested returns the work to Implementation.
- **QA outcome:** Pass goes on to release approval; fail creates a defect item and returns the work to Implementation.
- **Release decision:** Approved goes to release; rejected goes back to review or ends the instance.

2.6 Approvals and Dependencies

- **Approvals:** Code review needs one approving Reviewer, or two for high-risk features. Release needs the Tech Lead and then the Release Manager. Very low-risk changes (for example text changes) may be approved automatically.
- **Escalation:** If an approver does not respond within the agreed time (for example 24 hours), the request escalates to the Tech Lead.
- **Dependencies:** Frontend work depends on an agreed API contract from backend work. QA testing depends on implementation and code review being complete. Release approval depends on QA passing and documentation being finished.

2.7 Possible Failures and Exceptions

- The automated build or tests fail after a change is pushed.
- A reviewer rejects the work, or QA finds defects, sending the work back for rework (a loop).
- An approver is unavailable and the request times out.

- The feature request is cancelled while work is in progress, so all open items must be cancelled consistently.
- The Git hosting service is unreachable, or a merge conflict prevents the merge.
- A wrong action is taken (for example the wrong item is marked complete), so it must be undone, or the instance rolled back to an earlier checkpoint.

2.8 External Systems

- **Git hosting service:** Creates a branch for each development item, opens a pull request, requests a review and merges. Its interface is not designed for TIVIDY, so it is reached through a connector.
- **Build (CI) pipeline:** Reports build results back to the workflow as external events.
- **Messaging and email:** Notifies participants when items are assigned, approvals are requested or escalations happen.

2.9 Expected Outcome

A successful instance ends in the Completed state with the feature merged, released and documented, and the Requester notified. An unsuccessful instance ends in the Cancelled state with a recorded reason. In both cases TIVIDY keeps a complete audit history of who did what and when, and can report cycle time, current workload and delays or bottlenecks.

3. How the Scenario Uses TIVIDY

The scenario supplies the process, roles, rules and an external connector. TIVIDY supplies stages, work items, lifecycles, assignment, approvals, events, history and reporting. A different process, such as employee onboarding, would need a new definition, new roles and possibly a new connector, but no change to the core engine. This is the test that the scope in Section 1 is generic.